PYDICT++:

A PYTHON-INSPIRED DICTIONARY LIBRARY FOR C++ USING A DUAL-CHAIN HASH TABLE

(Single-Header C++17 Library)

A PROJECT REPORT

Submitted by

KARTIK BHARDWAJ (25BCS12463)
MANISH TANWAR (25BCS11666)
VINAY RAWAT (25BCS10360)

in partial fulfilment for the award of the degree of

BACHELOR OF ENGINEERING
IN
COMPUTER SCIENCE & ENGINEERING

Subject: Advanced Data Structures
Under the supervision of Mr. Tausef

CHANDIGARH UNIVERSITY
NOVEMBER 2026

BONAFIDE CERTIFICATE

Certified that this project report “PYDICT++: A PYTHON-INSPIRED DICTIONARY LIBRARY FOR C++ USING A DUAL-CHAIN HASH TABLE” is the bonafide work of “KARTIK BHARDWAJ (25BCS12463), MANISH TANWAR (25BCS11666) and VINAY RAWAT (25BCS10360)” who carried out the project work under my/our supervision.

SIGNATURE

HEAD OF THE DEPARTMENT
Computer Science & Engineering

SIGNATURE

SUPERVISOR
Mr. Tausef
Computer Science & Engineering

Submitted for the project viva-voce examination held on _____

INTERNAL EXAMINER

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

We would like to thank the Department of Computer Science & Engineering, Chandigarh University, for including a hands-on project in the Advanced Data Structures course. Building a complete container library, rather than solving isolated exercises on linked lists and hash tables, showed us how pointers, hashing, memory management and generic programming depend on each other.

We are grateful to our course faculty and project supervisor, Mr. Tausef, for explaining hashing, collision handling and amortised analysis with patience, and for his feedback on our design.

We also thank our classmates for trying the library on their own programs, and our families for their constant support during the semester.

KARTIK BHARDWAJ (25BCS12463)
MANISH TANWAR (25BCS11666)
VINAY RAWAT (25BCS10360)

B.E. CSE, Semester 3

ABSTRACT

Python’s dict is popular because it is easy to use: it remembers insertion order, offers one-line merge, pop and default-value operations, and prints readably. C++ programmers who want the same behaviour must combine std::unordered_map, which iterates in arbitrary order, with extra code written by hand. This project builds PyDict++, a single-header C++17 library that brings Python dictionary semantics to C++.

The library does not use std::unordered_map or std::map. Its engine is a custom hash table in which every node sits in two linked structures at once: a bucket chain for collision handling and a global doubly linked list that records insertion order. The table uses separate chaining, a prime-sized bucket array and a load-factor limit of 0.75; rehashing reuses each node’s stored hash value. On top of this engine we implemented bidirectional iterators, keys/values/items views, set-style operators (+, |, -), in-place sorting, a history engine with undo and redo, a JSON reader and writer, a nested heterogeneous dictionary type (PyValue), and stream input and output.

The header has 1,157 lines and compiles with g++ 13.3 using -std=c++17. It can be built in Visual Studio Code and, as a single bundled file, in an online compiler. Eight showcase scenarios and fifteen additional edge-case tests were run and their real output is reported. For one million integer keys, PyDict++ looked up values in 6.1 ms, close to std::unordered_map (4.7 ms) and about 18 times faster than std::map (113 ms). Inserts were slower (128 ms against 42 ms) because every insert also records history. The report also lists the limitations we found, such as JSON strings that contain quotation marks.

Keywords: hash table, separate chaining, dynamic rehashing, doubly linked list, iterators, undo/redo, C++17 templates, Python dictionary.

TABLE OF CONTENTS

BONAFIDE CERTIFICATE . . . . . **i**

ACKNOWLEDGEMENT . . . . . **ii**

ABSTRACT . . . . . **iii**

LIST OF FIGURES . . . . . **vi**

LIST OF TABLES . . . . . **vi**

CHAPTER 1. INTRODUCTION **1**

1.1 Background and Need for the Project 1

1.2 Why Both Visual Studio Code and an Online Compiler 1

1.3 Problem Statement 2

1.4 Objectives 2

1.5 Hardware and Software Used 3

1.6 Comparison with Existing Containers 3

1.7 Organisation of the Report 3

CHAPTER 2. DESIGN OF THE LIBRARY **4**

2.1 Overview of the Proposed Solution 4

2.2 The Dual-Chain Hash Table 4

2.3 Modules and Class Structure 5

2.4 Operations and Time Complexity 6

CHAPTER 3. WORKING OF THE LIBRARY **8**

3.1 Node and Hash Table Engine 8

3.2 Insertion 8

3.3 Erasure and Copying 9

3.4 Rehashing 10

3.5 Iterators and Views 10

3.6 History Engine (Undo and Redo) 10

3.7 JSON, Nested Values and Stream I/O 11
3.8 Public Interface 11

CHAPTER 4. IMPLEMENTATION 12
4.1 Project Files 12
4.2 Implementation in Visual Studio Code 12
4.3 Implementation in an Online Compiler (OnlineGDB) 13
4.4 Choosing Between the Two Environments 13

CHAPTER 5. RESULTS AND TESTING 14
5.1 Showcase Program Output 14
5.2 Edge-Case Tests 14
5.3 Hash Table Statistics 15
5.4 Performance Comparison 16
5.5 Limitations Found 17

CHAPTER 6. CONCLUSION AND FUTURE SCOPE 18
6.1 Conclusion 18
6.2 Future Scope 18

REFERENCES 19

LIST OF FIGURES

Figure 1.1: The gap in C++ that PyDict++ fills 1
Figure 2.1: One node, two chains: bucket chains (top) and the global insertion-order list (bottom) 5
Figure 2.2: Module structure of PyDict++ 6
Figure 3.1: Flow of an insert operation 9
Figure 4.1: Two ways to build the same project 12
Figure 5.1: Insert and lookup time for one million keys 16

LIST OF TABLES

Table 1.1: Hardware and software requirements 3
Table 1.2: Feature comparison 3
Table 2.1: Parts of the proposed solution 4
Table 2.2: Fields of a node (core::Node) 5
Table 2.3: Components of the library 6
Table 2.4: Time complexity of the main operations 7
Table 3.1: How each record type is undone and redone 10
Table 3.2: PyDict++ operations and their Python counterparts 11
Table 4.1: Source files of the project 12
Table 4.2: Visual Studio Code compared with an online compiler 13
Table 5.1: Edge-case tests and observed results 15
Table 5.2: Table state after N insertions 15
Table 5.3: Time in milliseconds (lower is better) 16

CHAPTER 1
INTRODUCTION

1.1 Background and Need for the Project

A dictionary maps keys to values, and almost every program needs one. In Python the built-in dict is easy to use: it keeps keys in the order they were inserted, merges with a single operator, supports pop, setdefault and get with a default value, and prints itself as {'a': 1} [6]. In C++ the closest container is std::unordered_map, which gives fast lookup but iterates in an unpredictable order, has no merge or difference operators, keeps no history and cannot be written to JSON [5]. The ordered std::map iterates in key order, but each lookup costs O(log n) [3].

This project is needed for three reasons. The first is convenience: Python-style operations such as get with a default, pop, popitem, setdefault, update, merge, difference and readable printing are missing from the C++ containers, so the same wrapper code has to be written again and again. The second is predictable order: an arbitrary iteration order makes output and debugging unpredictable, whereas Python keeps insertion order. The third is learning: ready-made containers hide how a hash table works, which is the main topic of the Advanced Data Structures course. Building the table from scratch lets every pointer and every rehash step be studied.

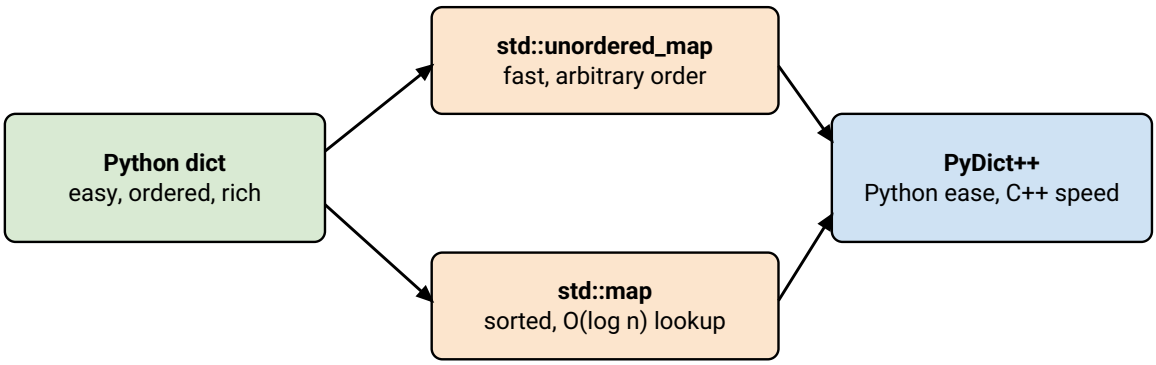


Figure 1.1: The gap in C++ that PyDict++ fills

1.2 Why Both Visual Studio Code and an Online Compiler

The library is a single header with no installation step and no third-party code, so it runs wherever a C++17 compiler is available. Visual Studio Code is the full development setup. It keeps the library and the demonstration program in separate files, provides build and launch settings, supports debugging, and uses the compiler installed on the student's own computer.

An online compiler such as OnlineGDB needs nothing installed. It is useful in a laboratory, on a borrowed computer, or for a quick demonstration during the viva. Because an online compiler normally accepts a single source file, the project also provides a bundled file in which the library and the demonstration program are combined. Supporting both environments means the project can be built, tested and shown on any device. Chapter 4 describes the steps for each.

1.3 Problem Statement

A programmer who wants Python-like dictionary behaviour in C++ has no single container that offers it. std::unordered_map has O(1) average lookup but no insertion order and no Python-style operations; std::map keeps an order, but by key and at O(log n) cost. The programmer therefore has to write wrapper code again and again, and such wrappers also hide how the hash table works, which is the main topic of this course. PyDict++ solves both problems: it builds the hash table from scratch, so every pointer and every rehash step can be studied, and it exposes a Python-style interface on top of it.

1.4 Objectives

- Build a hash table with separate chaining, a prime-sized bucket array and automatic rehashing, without using std::unordered_map or std::map.
- Keep insertion order by linking every node into a second, global doubly linked list, so that iteration is ordered and still O(n).
- Provide Python-style operations: get, pop, popitem, setdefault, update, keys/values/items views, merge (+, |) and difference (-).
- Add features that standard C++ containers lack: undo and redo, in-place sorting, JSON export and import, and nested dictionaries holding mixed types.
- Make the library easy to build in Visual Studio Code and in an online compiler.
- Verify every feature by compiling and running real programs, and measure the speed against std::unordered_map and std::map.

1.5 Hardware and Software Used

Table 1.1: Hardware and software requirements

Item	Specification
Language	C++17 (templates, std::variant, std::optional, structured bindings, if constexpr)
Compiler	GNU g++ 13.3.0 on Ubuntu 24.04, compiled with -std=c++17 -O2
Libraries	C++ standard library only; no third-party code
Source files	pydict.hpp (library, 1,157 lines), main.cpp (showcase, 91 lines), onlinegdb_bundle.cpp (both combined for the OnlineGDB compiler, 1,251 lines)
Local editor	Visual Studio Code (the project includes build and launch settings)
Online compiler	OnlineGDB, using the single bundled file
Hardware	Any laptop with at least 4 GB RAM; the library needs no installation because it is a single header

1.6 Comparison with Existing Containers

Table 1.2 compares PyDict++ with the Python dict it imitates and with the two standard C++ containers it is meant to replace. The first three columns reflect the documented behaviour of those containers; the last column describes what this project implements and tests.

Table 1.2: Feature comparison

Feature	Python dict	unordered_map	std::map	PyDict++
Iteration order	Insertion	Arbitrary	Sorted by key	Insertion
Average lookup	O(1)	O(1)	O(log n)	O(1)
Merge operator	Yes (|)	No	No	Yes (+, |)
Undo / redo	No	No	No	Yes
JSON output	json module	No	No	Built in
Prints as {'a': 1}	Yes	No	No	Yes

1.7 Organisation of the Report

Chapter 2 explains the design of the solution: the dual-chain node, the three modules and the time complexity of each operation. Chapter 3 explains how the library works internally, step by step. Chapter 4 gives the way to implement the solution in Visual Studio Code and in an online compiler. Chapter 5 reports the results of the showcase program, fifteen edge-case tests, table statistics and a speed comparison, together with the limitations we found. Chapter 6 concludes and lists future work.

CHAPTER 2
DESIGN OF THE LIBRARY

2.1 Overview of the Proposed Solution

PyDict++ is a single-header C++17 library (pydict.hpp, 1,157 lines) that uses only the C++ standard library. Its central idea is a hash table in which every element is stored in one node that belongs to two lists at the same time: a bucket chain for fast lookup and a global list for insertion order. On this engine the library adds a Python-style interface and several extra features. Table 2.1 summarises the parts of the solution.

Table 2.1: Parts of the proposed solution

Part	What it provides
Hash table engine	Separate chaining, prime-sized bucket array, load-factor limit 0.75, rehashing with stored hash values, statistics (printStats)
Insertion order	A global doubly linked list inside the same nodes; iteration never skips empty buckets
Python-style interface	get, pop, popitem, setdefault, update, the operators +, | and -, keys/values/items views, readable printing
Extra features	Undo and redo history, sortByKey and sortByValue, JSON save and load, nested dictionaries (PyValue), stream input and output
Error handling	PyDictException with KeyError, ValueError, HistoryEmptyError and SerializationError, using Python-like messages

2.2 The Dual-Chain Hash Table

Each element is stored in one node. The first list is the bucket chain: the node's hash value, taken modulo the number of buckets, selects a bucket, and nodes that land in the same bucket are joined with next_in_bucket (separate chaining) [1]. The second list is the global list: all nodes are joined in insertion order with prev_global and next_global, and the table keeps head and tail pointers. Lookup walks only the bucket chain. Iteration walks only the global list, so it never has to skip empty buckets. Figure 2.1 shows both lists; the same colour marks the same node in both.

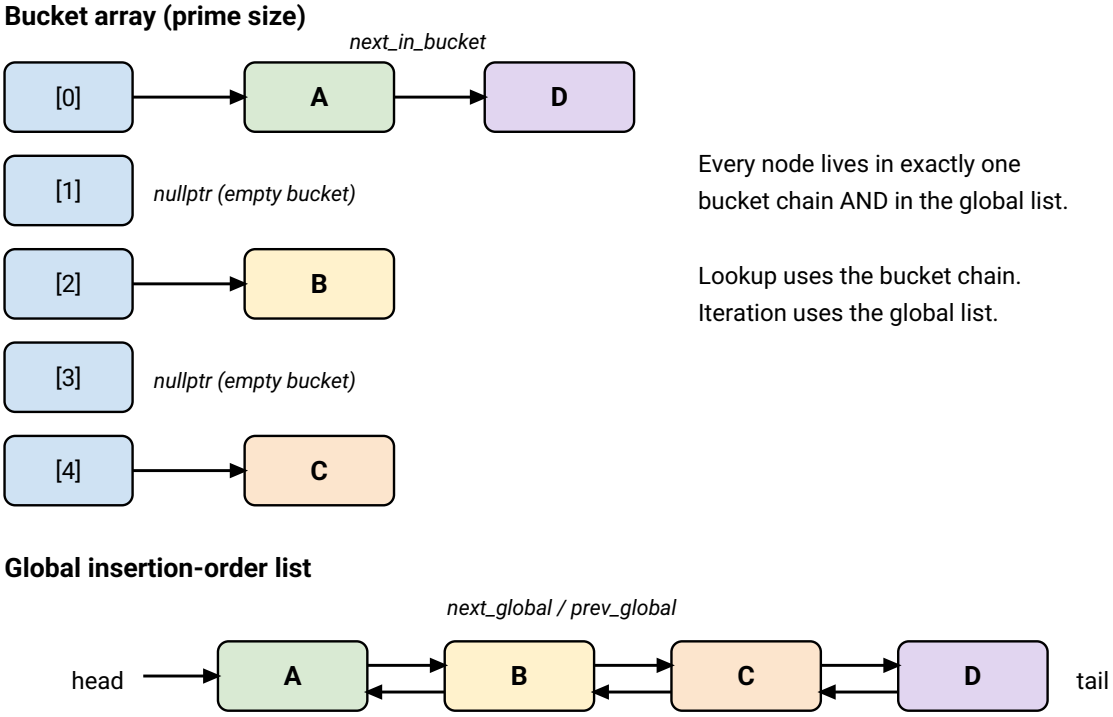


Figure 2.1: One node, two chains: bucket chains (top) and the global insertion-order list (bottom)

Table 2.2: Fields of a node (core::Node)

Field	Purpose
key, value	The stored pair (copied or moved in)
hash_val	Hash computed once and stored; compared before the key itself, and reused during rehashing
next_in_bucket	Next node in the same bucket (collision chain)
prev_global, next_global	Neighbours in the insertion-order list; allow O(1) unlinking and backward iteration

Because the node copy constructor and assignment operator are deleted, a node can never be duplicated by accident, and each node is owned by exactly one table. The table deletes nodes by walking the global list, so no node is leaked.

2.3 Modules and Class Structure

The header is divided into three namespaces, matching the three parts of the work shared among the team: pydictpp::core (hash table engine), pydictpp::dict (iterators and views) and pydictpp::features (history and JSON). The user-facing class PyDict inherits publicly from core::HashTable and adds the Python-style interface. Figure 2.2 shows the structure and Table 2.3 lists the components.

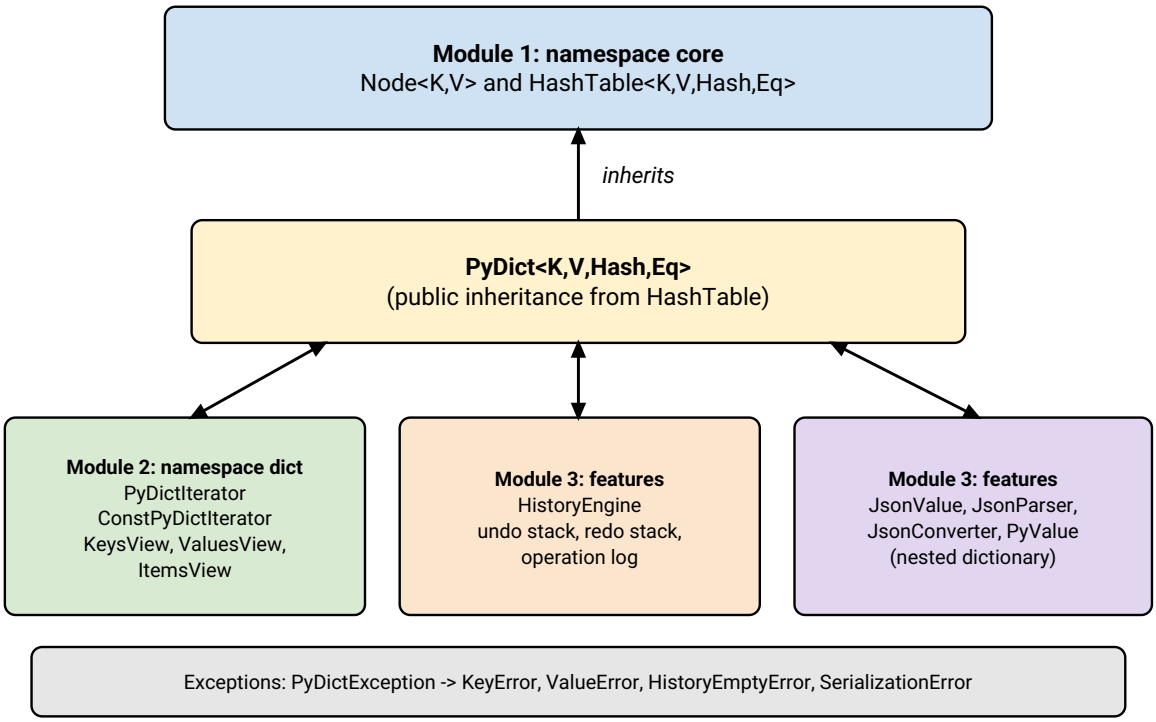


Figure 2.2: Module structure of PyDict++

Table 2.3: Components of the library

Module	Components	Role
Module 1: core	Node, HashTable, prime list	Storage, hashing, collision chains, rehashing, statistics (printStats)
Module 2: dict	PyDictIterator, ConstPyDictIterator, KeysView, ValuesView, ItemsView	Bidirectional iteration and lightweight views that print like Python's dict_keys, dict_values and dict_items
Module 3: features	HistoryRecord, HistoryEngine, JsonValue, JsonParser, JsonConverter, PyValue	Undo and redo, JSON text, and nested dictionaries holding bool, int, double, string, list or dictionary
Errors	PyDictException and four subclasses	KeyError, ValueError, HistoryEmptyError, SerializationError, with Python-like messages

2.4 Operations and Time Complexity

Table 2.4 gives the cost of the main operations, derived from the code in pydict.hpp. Average-case costs assume that the hash function spreads keys evenly; a poor hash function would make the chains long and push the worst case towards O(n) [2].

Table 2.4: Time complexity of the main operations

Operation	Average	Notes
insert, operator[], contains, get	O(1)	Worst case O(n) if every key lands in one bucket
erase, pop	O(1)	Unlinks from both the bucket chain and the global list
popitem	O(1)	Removes the most recently inserted key (tail)
rehash	O(n)	Amortised O(1) per insert because capacity roughly doubles
iteration, keys/values/items, update	O(n)	Walks the global list; views are created in O(1)
sortByKey, sortByValue	O(n log n)	std::sort on node pointers, then relinking the global list
findValue	O(n)	Values are not indexed
undo, redo	O(1)	O(n) when the step is a clear, because all pairs are restored

CHAPTER 3
WORKING OF THE LIBRARY

3.1 Node and Hash Table Engine

HashTable stores a vector of node pointers (buckets_), the head_ and tail_ of the global list, the element count, the maximum load factor (0.75) and a counter of collisions. The constructor accepts an initial capacity and rounds it up to the next prime from a fixed list of 23 primes (17 up to 67,108,879). Lookup compares the stored hash first and calls the key-equality function only when the hashes match:

```
NodeType* find_node(const Key& key) const {
    size_t h = compute_hash(key);
    NodeType* curr = buckets_[bucket_index(h)];
    while (curr != nullptr) {
        if (curr->hash_val == h && equal_fn_(curr->key, key)) return curr;
        curr = curr->next_in_bucket;
    }
    return nullptr;
}
```

3.2 Insertion

Insertion first checks the load factor, then searches the chain. If the key exists, its value is overwritten and its position in the order is unchanged. Otherwise a new node is placed at the head of the bucket chain, which takes O(1) time, and appended to the tail of the global list. A collision is counted whenever the target bucket was not empty. Figure 3.1 shows the flow and the code below shows the linking step.

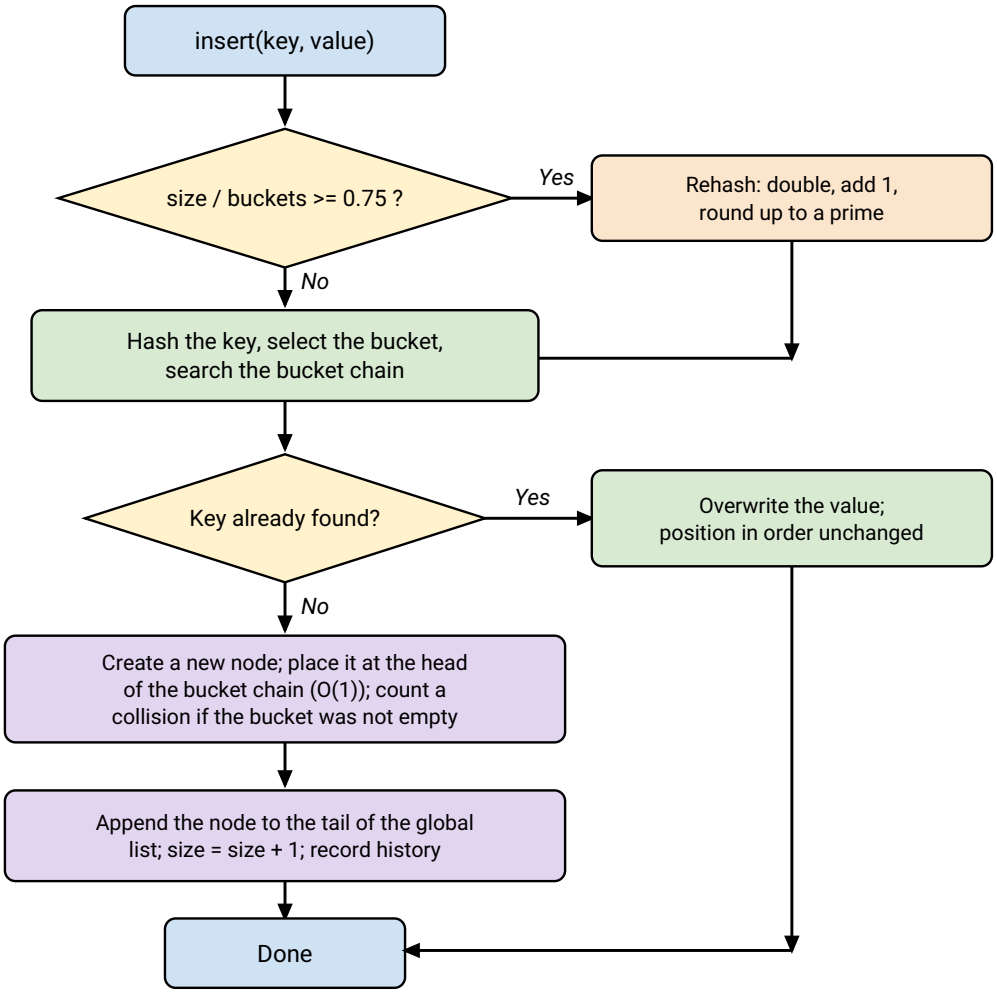


Figure 3.1: Flow of an insert operation

```
if (buckets_[idx] != nullptr) ++collision_count_;
NodeType* n = new NodeType(std::forward<K>(key), std::forward<V>(value), h);
n->next_in_bucket = buckets_[idx];  buckets_[idx] = n;
if (!tail_) { head_ = tail_ = n; }
else { tail_->next_global = n; n->prev_global = tail_; tail_ = n; }
++size_;
```

3.3 Erasure and Copying

Erasing finds the node while remembering its predecessor in the bucket chain, then fixes the bucket chain and the global list, updating head_ or tail_ when needed, before deleting the node. The copy constructor re-inserts nodes in global order, so a copied dictionary keeps the same order.

3.4 Rehashing

Before every insert the table checks whether size divided by the number of buckets has reached the maximum load factor of 0.75. If so, the bucket count is doubled plus one and rounded up to the next prime. Rehashing walks the global list and moves each node into the new bucket array using its stored hash_val, so no key is hashed again and no node is allocated or copied. Prime sizes help because the bucket index is the hash modulo the bucket count: with std::hash, which returns the number itself, a prime modulus avoids the clustering that a power of two would cause for keys that are multiples of a common factor [3]. Section 5.3 shows the bucket counts that result.

3.5 Iterators and Views

PyDictIterator and ConstPyDictIterator are bidirectional iterators that follow next_global and prev_global. They make range-based loops such as for (const auto& [key, value] : dict) work and visit pairs in insertion order. KeysView, ValuesView and ItemsView hold only the head pointer and the size, so creating a view costs O(1); each can be converted to a std::vector or printed in the Python style, for example dict_keys([apples, milk, bread]).

3.6 History Engine (Undo and Redo)

Every changing operation (insert, update, erase, clear) creates a HistoryRecord holding the key, the old and new values, a timestamp and, for clear, a snapshot of all pairs. HistoryEngine keeps three vectors: an undo stack, a redo stack and a full log used by printHistory. A new operation empties the redo stack, as in a text editor. While an undo or redo is running, recording is switched off so that the undo itself is not logged; it uses insert_or_assign, which never writes history. Table 3.1 shows how each record type is reversed.

Table 3.1: How each record type is undone and redone

Record	Undo	Redo
INSERT	erase(key)	insert_or_assign(key, new value)
UPDATE	insert_or_assign(key, old value)	insert_or_assign(key, new value)
ERASE	insert_or_assign(key, old value)	erase(key)
CLEAR	re-insert every pair of the snapshot	clear()

3.7 JSON, Nested Values and Stream I/O

JsonValue is a small tagged value (null, boolean, integer, number, string, array, object) with a dump method that prints indented text. JsonParser is a recursive-descent parser that throws SerializationError on malformed input, and JsonConverter converts between C++ types and JsonValue using if constexpr. The dictionary offers toJSONString, fromJSONString, saveToJSON and loadFromJSON.

PyValue is a std::variant over none, bool, 64-bit integer, double, string, list and a shared pointer to a dictionary [4]. Because a PyDict can hold PyValue objects, dictionaries nest to any depth, and PyAnyDict (a dictionary from string to PyValue) allows code such as config["server"]["port"] = 8080 in which missing levels are created on first use. Stream output prints Python syntax ('key': value), and operator>> reads the same syntax back.

3.8 Public Interface

Table 3.2: PyDict++ operations and their Python counterparts

PyDict++	Python	Behaviour
insert(k, v), d[k] = v	d[k] = v	Adds or updates; insert returns true only for a new key
get(k, default)	d.get(k, default)	Never throws
pop(k), pop(k, default)	d.pop(k[, default])	Throws KeyError without a default
popitem()	d.popitem()	Removes the last inserted pair
setdefault(k, v)	d.setdefault(k, v)	Inserts only if the key is absent
update(other), +, |, +=	d.update(o), d | o	Merge; right-hand values win
a - b, -=	(dict comprehension)	Pairs of a whose key is not in b
keys(), values(), items()	d.keys() and so on	Views in insertion order
undo(), redo(), printHistory()	(none)	Added feature
sortByKey(), sortByValue()	dict(sorted(...))	Sorts in place by relinking nodes
toJSONString(), saveToJSON()	json.dumps, json.dump	Added feature

CHAPTER 4
IMPLEMENTATION

4.1 Project Files

The project consists of three source files. The same library is used in both environments; only the packaging differs, as shown in Figure 4.1.

Table 4.1: Source files of the project

File	Lines	Use
pydict.hpp	1,157	The library (single header)
main.cpp	91	Showcase program with eight feature groups; includes pydict.hpp
onlinegdb_bundle.cpp	1,251	Library and showcase program combined into one file for online compilers

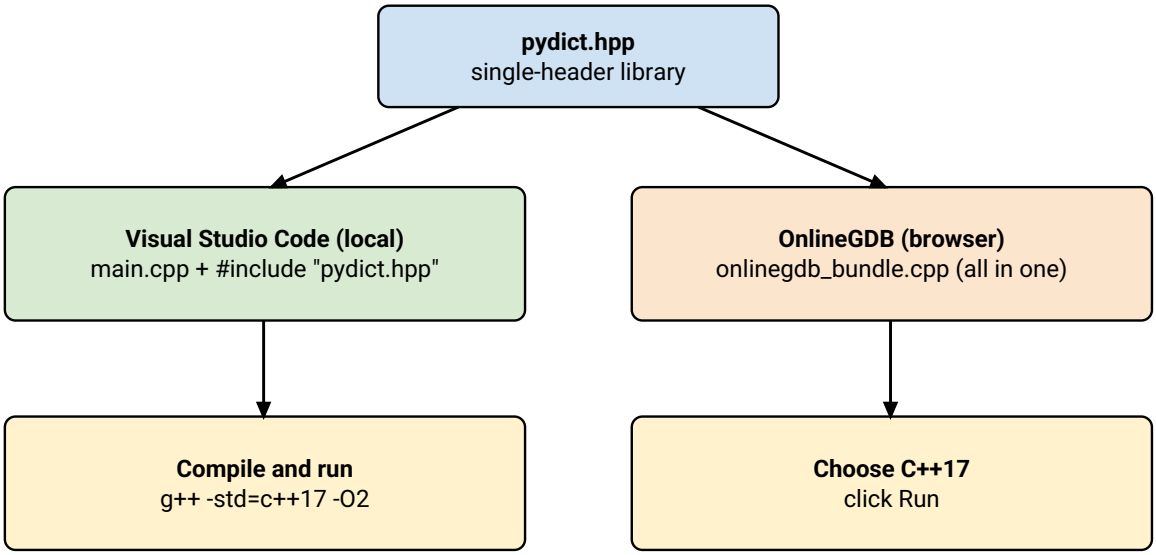


Figure 4.1: Two ways to build the same project

4.2 Implementation in Visual Studio Code

1. **Install a compiler.** The project was built with GNU g++ 13.3.0 on Ubuntu 24.04, but any g++ that supports C++17 will do. Check the installation with g++ --version. Install the C/C++ extension in Visual Studio Code for IntelliSense and debugging.
2. **Open the project folder** (File, Open Folder) that contains pydict.hpp and main.cpp. The project includes the Visual Studio Code build and launch settings.
3. **Build the program** with the build task (Terminal, Run Build Task) or with the following command in the integrated terminal:

```
g++ -std=c++17 -O2 main.cpp -o pydict_demo
```

4. **Run the program** from the terminal. The eight showcase scenarios of Section 5.1 are printed.

```
./pydict_demo        (Linux, macOS)
pydict_demo.exe      (Windows)
```

5. **Use the library in your own program.** Copy pydict.hpp next to your source file and include it. No linking and no installation is needed:

```
#include "pydict.hpp"
using namespace pydictpp;   // PyDict<K,V> can now be used as in Chapter 3
```

6. **Debug.** Use the launch configuration to set breakpoints, for example inside find_node or the rehash step, and watch the pointers change.

4.3 Implementation in an Online Compiler (OnlineGDB)

1. **Open** the online C++ compiler in a web browser. Nothing needs to be installed.
2. **Select the language C++17** in the language list. The library needs C++17 because it uses std::variant, std::optional, structured bindings and if constexpr.
3. **Paste the whole content of onlinegdb_bundle.cpp** into the editor. An online compiler holds a single file, so the header is already merged with the showcase program.
4. **Click Run.** The eight showcase scenarios appear in the output window.
5. **Give the input.** Scenario 8 reads a dictionary from the keyboard (cin >> dict). When the program waits for input, type the dictionary in Python syntax, for example {'book': 15, 'pen': 3}.

4.4 Choosing Between the Two Environments

Table 4.2: Visual Studio Code compared with an online compiler

Aspect	Visual Studio Code	Online compiler
Installation	Compiler and editor are needed	None; runs in a browser
Files	pydict.hpp and main.cpp	Single file onlinegdb_bundle.cpp
Best used for	Development, debugging, larger programs	Quick demonstration, laboratory or viva, any device
Timing measurements	Reliable with -O2 on the local machine	Shared server, so times vary

CHAPTER 5
RESULTS AND TESTING

All results in this chapter come from compiling the library with g++ 13.3.0 (-std=c++17 -O2) and running the programs; nothing is copied from documentation. The compiler printed three -Wmisleading-indentation warnings for lines in pydict.hpp where an if statement and the next statement share one line (lines 487, 515 and 1049); they do not affect behaviour.

5.1 Showcase Program Output

main.cpp exercises eight feature groups. The output below is from the real run (the banner lines and the JSON text block are shortened); every line matched the expected Python behaviour.

```
1. Core Operations:
  Dictionary: {'apples': 2.99, 'milk': 3.49, 'bread': 2.25, 'eggs': 4.19}
  Contains 'milk': Yes
  Safe get('cheese', 0.0): $0
  Popped last item: eggs
2. Views & Iteration in Insertion Order:
  Keys:   dict_keys([apples, milk, bread])
  Values: dict_values([2.99, 3.49, 2.25])
  Items:  dict_items([(apples, 2.99), (milk, 3.49), (bread, 2.25)])
3. Merge (+) and Difference (-):
  Merged:     {'laptops': 10, 'keyboards': 50, 'monitors': 15}
  Difference: {'laptops': 10}
4. Undo / Redo System:
  Before undo: {'score': 250, 'level': 1}
  After undo:  {'score': 100, 'level': 1}
  After redo:  {'score': 250, 'level': 1}
5. In-Place Sorting:
  Sorted by key:   {'Alice': 98, 'Bob': 74, 'Charlie': 82}
  Sorted by value: {'Alice': 98, 'Charlie': 82, 'Bob': 74}
6. JSON Round-Trip:
  Restored from JSON: {'Alice': 98, 'Charlie': 82, 'Bob': 74}
7. Dynamic Nested Dictionary:
  Config: {'server': {'host': '127.0.0.1', 'port': 8080}, 'active': True}
  Host: 127.0.0.1
8. Stream I/O:
  Read from stream (cin >> dict): {'book': 15, 'pen': 3}
```

5.2 Edge-Case Tests

Fifteen further tests were written to probe error handling and corner cases. Each produced the expected result, as listed in Table 5.1.

Table 5.1: Edge-case tests and observed results

#	Test	Observed result
T1	pop() of a missing key	KeyError: 'zz'
T2	const operator[] on a missing key	KeyError: 'zz'
T3	popitem() on an empty dictionary	KeyError: 'popitem(): dictionary is empty'
T4	undo() with no history	HistoryEmptyError: No operations in history
T5	redo() after a new operation followed an undo	HistoryEmptyError (redo stack was cleared)
T6	Erase key 3, then insert it again	{1, 2, 4, 5, 6, 3}: key 3 moves to the end
T7	clear(), then undo()	{} then {1: 1, 2: 2}: all pairs restored
T8	Modify a copy made with copy()	Original {1: 1}, copy {1: 1, 2: 2}: independent
T9	pop("q", -1) for a missing key	Returns -1 without throwing
T10	findValue(1) with x and y both mapped to 1	x, y
T11	setdefault("k", 5) then setdefault("k", 9)	{'k': 5}: second call does not overwrite
T12	Compare {1:1, 2:2} with {2:2, 1:1}	Equal: comparison ignores order
T13	fromJSONString with truncated text	SerializationError: Unterminated JSON string
T14	loadFromJSON of a file that does not exist	SerializationError: Could not read: ...
T15	Undo after erasing key 2 from {1..5}	{1, 3, 4, 5, 2}: restored key goes to the end

5.3 Hash Table Statistics

The library's own counters (bucketCount, loadFactor, collisionCount, maxBucketDepth, emptyBucketCount) were read after inserting sequential integer keys and, separately, 5,000 string keys of the form key0 to key4999. Table 5.2 shows the result.

Table 5.2: Table state after N insertions

Keys	Elements	Buckets	Load factor	Collisions	Max chain
int	10	17	0.588	0	1
int	100	257	0.389	0	1
int	1,000	2,053	0.487	0	1
int	10,000	32,771	0.305	0	1
int	100,000	524,309	0.191	0	1
string	5,000	8,209	0.609	1,704	5

Every bucket count is a prime from the built-in list and the load factor never exceeded 0.75. Sequential integers hash to themselves under std::hash, so they spread perfectly and gave no collisions. The 5,000 string keys produced 1,704 collisions, but the longest chain held only five nodes, so lookups stay close to constant time.

5.4 Performance Comparison

A single timing run measured inserting and then looking up n sequential integer keys in PyDict++, std::unordered_map and std::map. Times were taken with std::chrono::steady_clock on the Ubuntu machine described in Chapter 1; they vary between runs and are meant to show orders of magnitude, not exact figures.

Table 5.3: Time in milliseconds (lower is better)

n	Operation	PyDict++	unordered_map	std::map
100,000	insert	11.9	2.8	10.2
100,000	lookup	0.30	0.35	6.9
1,000,000	insert	128.1	42.3	191.5
1,000,000	lookup	6.1	4.7	113.0

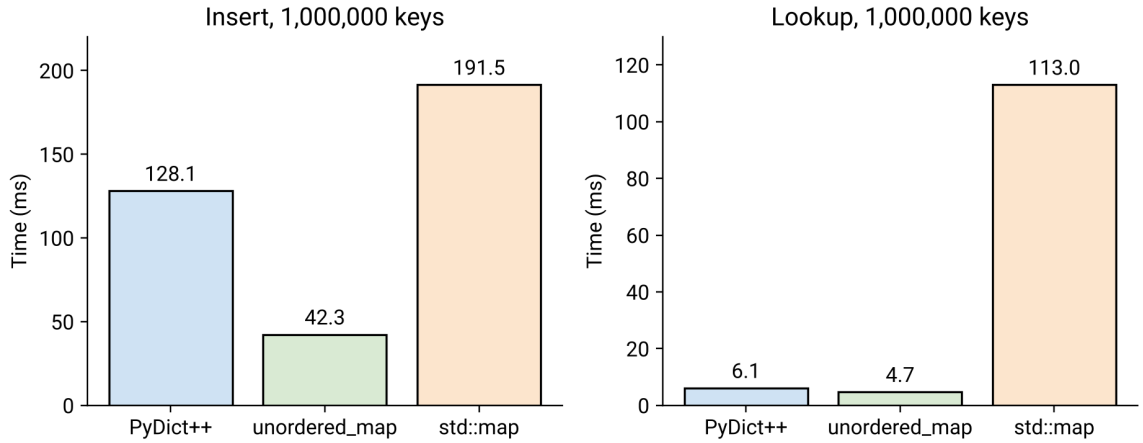


Figure 5.1: Insert and lookup time for one million keys

At one million keys, lookup in PyDict++ is within about 30% of std::unordered_map (at 100,000 keys it was slightly faster) and far faster than std::map. Insert is slower than std::unordered_map. A follow-up test of one million inserts, repeated three times, shows that most of the gap comes from the history feature: with history on, the runs took 140, 103 and 97 ms; after calling enableHistory(false) they took 79, 57 and 52 ms. Disabling history is therefore advisable for large bulk loads.

5.5 Limitations Found

Testing also revealed behaviour that users should know about.

- **JSON strings are not escaped.** A value such as he said "hi" is written with its inner quotation marks unchanged, and reading it back throws SerializationError. Only simple strings round-trip safely.
- **Sorting is not undoable.** sortByKey and sortByValue relink the global list without recording history, so a following undo() reverses the last insert, not the sort.
- **Undo does not restore position.** An erased key brought back by undo() is appended at the end of the order, not at its old place.
- **History grows without limit.** The log keeps every record for the life of the object, which costs memory and, as measured above, time.
- **No synchronisation.** The code contains no locks, so one dictionary must not be modified from several threads at once.

CHAPTER 6
CONCLUSION AND FUTURE SCOPE

6.1 Conclusion

PyDict++ shows that a Python-style dictionary can be built in C++ from basic data structures. A hash table with separate chaining gives constant-time lookup, a second doubly linked list inside the same nodes gives insertion order without extra memory for a separate index, and prime-sized rehashing keeps chains short: the longest chain in our tests was five nodes. The three-module design kept the work divisible among the team and let each part be tested alone.

Real runs confirmed that all eight showcase scenarios and fifteen edge-case tests behave as intended, that lookups are close to std::unordered_map and much faster than std::map, and that history recording is the main cost of inserts. Because the library is a single header, it builds in Visual Studio Code and, in its bundled form, in an online compiler without any installation. We also documented the places where the library falls short, so that its limits are as clear as its features.

6.2 Future Scope

- Escape and unescape quotation marks, backslashes and control characters in the JSON writer and reader.
- Record sort operations in the history, remember each key’s original position, and offer a cap on history length.
- Add move-only inserts and emplace to avoid copies, and a reserve() function to size the table in advance.
- Replace chaining at long buckets with a balanced tree, or use open addressing, and compare their speed with the current design.
- Add a thread-safe wrapper and a test suite built on a standard unit-testing framework.

REFERENCES

[1] T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, *Introduction to Algorithms*, 4th ed. Cambridge, MA, USA: MIT Press, 2022.

[2] D. E. Knuth, *The Art of Computer Programming, Vol. 3: Sorting and Searching*, 2nd ed. Reading, MA, USA: Addison-Wesley, 1998.

[3] M. T. Goodrich, R. Tamassia and D. M. Mount, *Data Structures and Algorithms in C++*, 2nd ed. Hoboken, NJ, USA: Wiley, 2011.

[4] ISO/IEC 14882:2017, *Programming Languages — C++*, International Organization for Standardization, 2017.

[5] cppreference.com, “std::unordered_map.” [Online]. Available: https://en.cppreference.com/w/cpp/container/unordered_map

[6] Python Software Foundation, “Mapping Types — dict,” *The Python Standard Library*. [Online]. Available: https://docs.python.org/3/library/stdtypes.html#mapping-types-dict